

PYTHON ESSENTIALS – EVALUATED COURSE PROJECT

TIC-TAC-TOE GAME

Submitted By	Anshika Trivedi
Registration Number	26BCE10853
Course	Python Essentials
Program	B.Tech CSE
University	VIT Bhopal University
Academic Year	2026–27
Project Type	Python Console-Based Game

A beginner-level console game demonstrating fundamental Python programming concepts.

1. Introduction

Tic-Tac-Toe is a simple strategy-based game played on a 3×3 board. Two players use X and O and take turns placing their symbols. A player wins by getting three matching symbols in a horizontal, vertical, or diagonal line. If all positions are occupied without a winner, the game ends in a draw.

This project implements a console-based Tic-Tac-Toe game using Python. It provides Two Players and Player vs Computer modes and applies lists, tuples, variables, conditions, loops, user input, indexing, and type conversion.

2. Objectives

- Develop a functional game.
- Create and display a 3×3 board.
- Provide two game modes.
- Validate positions.
- Detect winners and draws.
- Allow replay.

3. Problem Statement

Develop a simple and interactive Tic-Tac-Toe game using Python. The program should allow users to select a mode, enter positions, update the board, detect winners or draws, and start another game.

4. Functional Requirements

- Game Mode Selection
- Board Display
- Player Input
- Position Validation
- Computer Move
- Winner Detection
- Draw Detection
- Play Again

5. Non-Functional Requirements

- Simple and easy to use.
- Quick response to input.
- Reliable for supported game conditions.
- Understandable code.
- No external Python libraries required.

6. System Architecture

User → Input → Game Board → Game Logic → Result

User input is received, the board is updated, game logic is processed, and the result is displayed.

7. Use Case Diagram

Actors: Player 1, Player 2, Computer

Use cases: Select Game Mode, Enter Position, View Board, Make Computer Move, Check Winner, Check Draw, Play Again.

8. Workflow Diagram

START → Select Game Mode → Display Board → Make Move → Check Winner/Draw → Continue Turn or End Game → Play Again? → END

9. Sequence Diagram

Player selects mode → Program receives input → Board is initialized → Player enters position → Program validates and updates board → Result is checked → Result is displayed.

10. Component Diagram

The procedural program is divided logically into User Interface, Game Control, Move Handling, Winner/Draw Checking, and Board Data.

11. ER Diagram

No database is required because permanent data storage is not needed. Conceptually: **GAME → BOARD → POSITION (1–9)**.

12. Design Decisions

A Python list is used for the board. Positions 1–9 make selection easy. X and O represent players. Loops repeat turns. Tuples store winning combinations. A console interface keeps the project focused on Python fundamentals.

13. Implementation

The project was implemented and tested in Jupyter Notebook. Main sections include game initialization, mode selection, player movement, computer movement, winner checking, draw checking, and play-again functionality.

Python concepts used: variables, strings, lists, tuples, if-else, while loops, for loops, input(), int(), indexing, and comparison/logical operators.

14. Screenshots and Results

The following screenshots were taken from the actual execution of the project in Jupyter Notebook.

15. Testing

The project was tested using Two Players mode, Player vs Computer mode, valid positions, occupied positions, invalid positions, winning combinations, draw condition, and play-again functionality.

16. Challenges Faced

Challenges included mapping positions to list indexes, organizing eight winning combinations, handling two game modes, implementing the draw condition, integrating all parts, and testing different inputs.

17. Learning Outcomes

The project improved understanding of lists, indexing, loops, conditions, input and type conversion, debugging, testing, and combining basic Python concepts into a complete application.

18. Future Enhancements

Possible enhancements include difficulty levels, improved computer strategy, a graphical interface, score tracking, player names, sound effects, themes, and score storage.

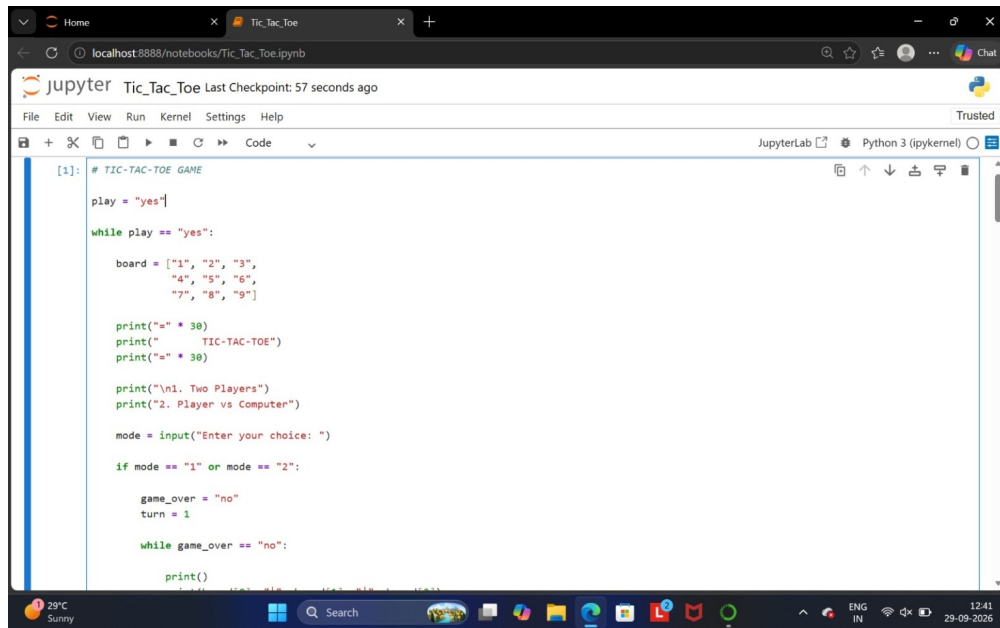
19. Conclusion

The project demonstrates basic Python programming concepts in a practical application. It provides two game modes and includes move handling, position validation, winner detection, draw detection, and replay functionality.

20. References

1. Python Essentials course material and class notes.
2. Python documentation for basic Python syntax and built-in functions.
3. Practice exercises and examples covered during the Python Essentials course.

Screenshots and Results



```
[1]: # TIC-TAC-TOE GAME

play = "yes"

while play == "yes":

    board = ["1", "2", "3",
             "4", "5", "6",
             "7", "8", "9"]

    print("\n * 30")
    print("    TIC-TAC-TOE")
    print("\n * 30")

    print("\n1. Two Players")
    print("2. Player vs Computer")

    mode = input("Enter your choice: ")

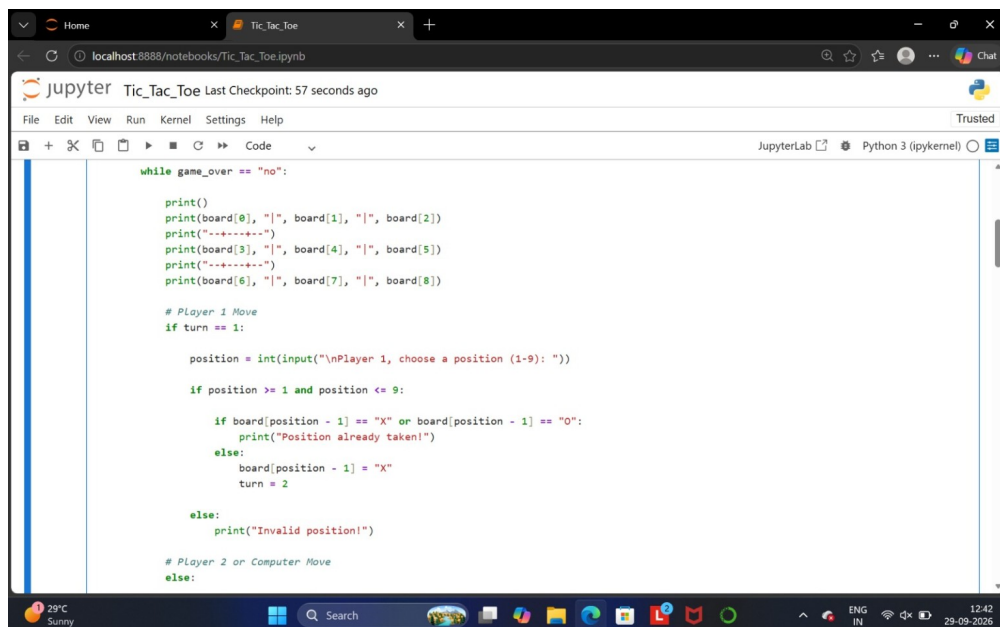
    if mode == "1" or mode == "2":

        game_over = "no"
        turn = 1

        while game_over == "no":

            print()
```

Figure 1. Project screenshot from Jupyter Notebook.



```
while game_over == "no":

    print()
    print(board[0], "|", board[1], "|", board[2])
    print("-----")
    print(board[3], "|", board[4], "|", board[5])
    print("-----")
    print(board[6], "|", board[7], "|", board[8])

    # Player 1 Move
    if turn == 1:

        position = int(input("\nPlayer 1, choose a position (1-9): "))

        if position >= 1 and position <= 9:

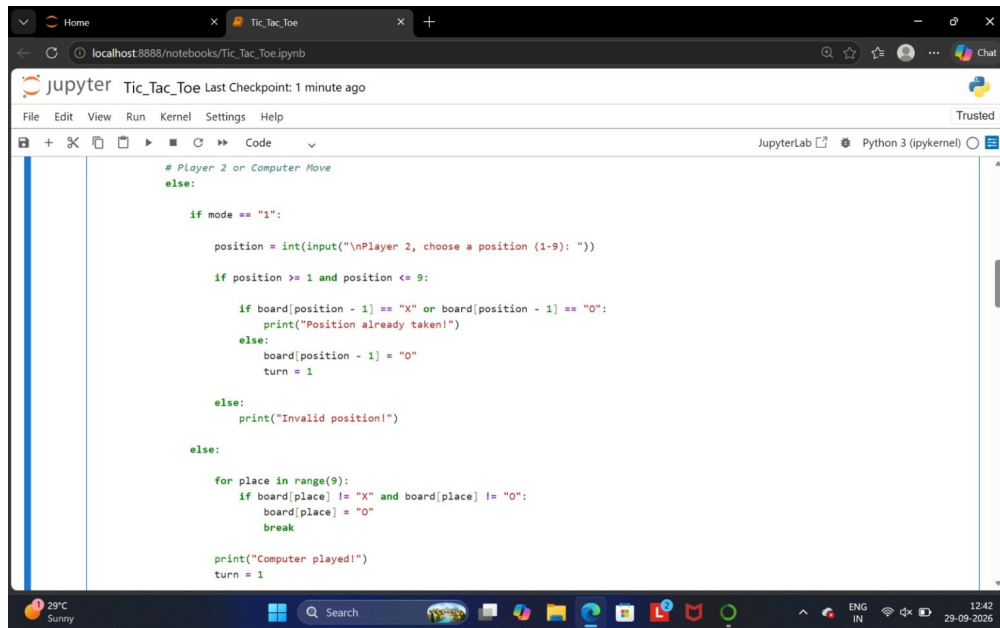
            if board[position - 1] == "X" or board[position - 1] == "O":
                print("Position already taken!")
            else:
                board[position - 1] = "X"
                turn = 2

        else:
            print("Invalid position!")

    # Player 2 or Computer Move
    else:
```

Figure 2. Project screenshot from Jupyter Notebook.

Screenshots and Results



A screenshot of a Jupyter Notebook interface. The browser address bar shows 'localhost:8888/notebooks/Tic_Tac_Toe.ipynb'. The notebook title is 'Tic_Tac_Toe' and it shows 'Last Checkpoint: 1 minute ago'. The code in the cell is as follows:

```
# Player 2 or Computer Move
else:
    if mode == "1":
        position = int(input("\nPlayer 2, choose a position (1-9): "))

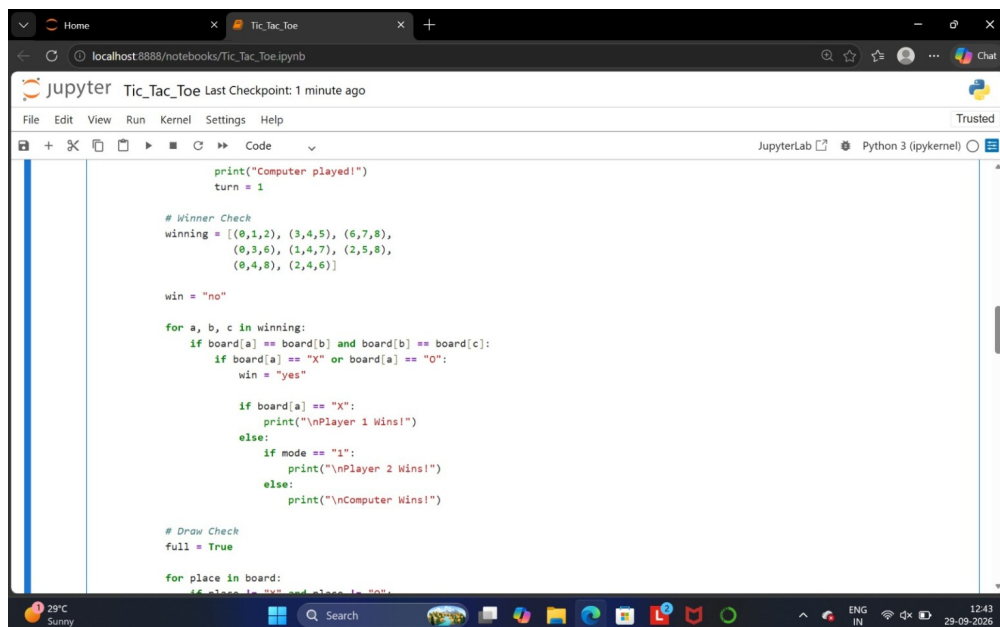
        if position >= 1 and position <= 9:
            if board[position - 1] == "X" or board[position - 1] == "O":
                print("Position already taken!")
            else:
                board[position - 1] = "O"
                turn = 1

        else:
            print("Invalid position!")

    else:
        for place in range(9):
            if board[place] != "X" and board[place] != "O":
                board[place] = "O"
                break

        print("Computer played!")
        turn = 1
```

Figure 3. Project screenshot from Jupyter Notebook.



A screenshot of a Jupyter Notebook interface, continuing from the previous one. The code in the cell is as follows:

```
print("Computer played!")
turn = 1

# Winner Check
winning = [(0,1,2), (3,4,5), (6,7,8),
           (0,3,6), (1,4,7), (2,5,8),
           (0,4,8), (2,4,6)]

win = "no"

for a, b, c in winning:
    if board[a] == board[b] and board[b] == board[c]:
        if board[a] == "X" or board[a] == "O":
            win = "yes"

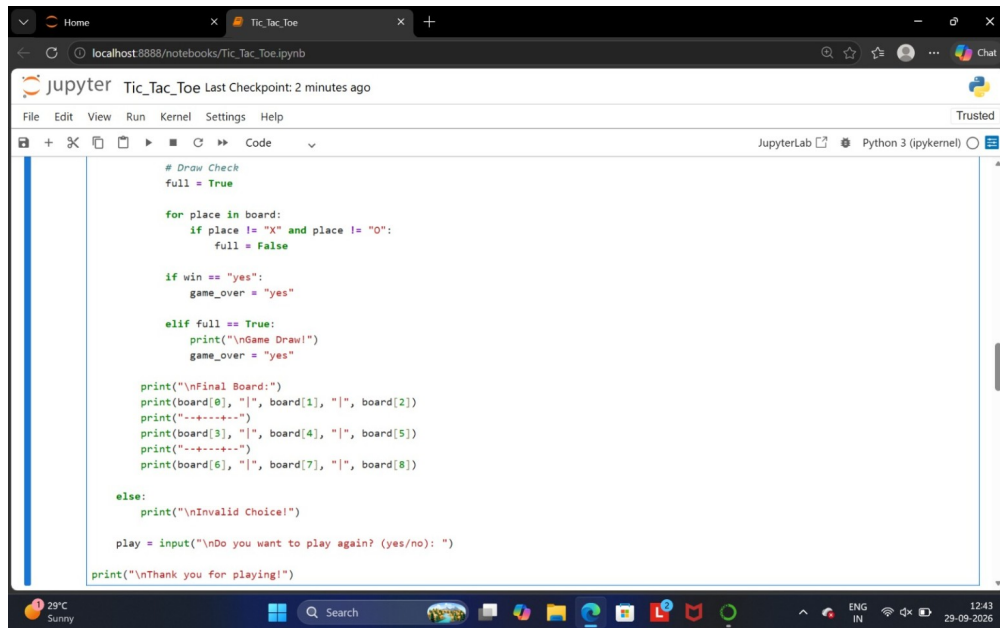
            if board[a] == "X":
                print("\nPlayer 1 Wins!")
            else:
                if mode == "1":
                    print("\nPlayer 2 Wins!")
                else:
                    print("\nComputer Wins!")

# Draw Check
full = True

for place in board:
    if place != "X" and place != "O":
        full = False
```

Figure 4. Project screenshot from Jupyter Notebook.

Screenshots and Results



```
# Draw Check
full = True

for place in board:
    if place != "X" and place != "O":
        full = False

if win == "yes":
    game_over = "yes"

elif full == True:
    print("\nGame Draw!")
    game_over = "yes"

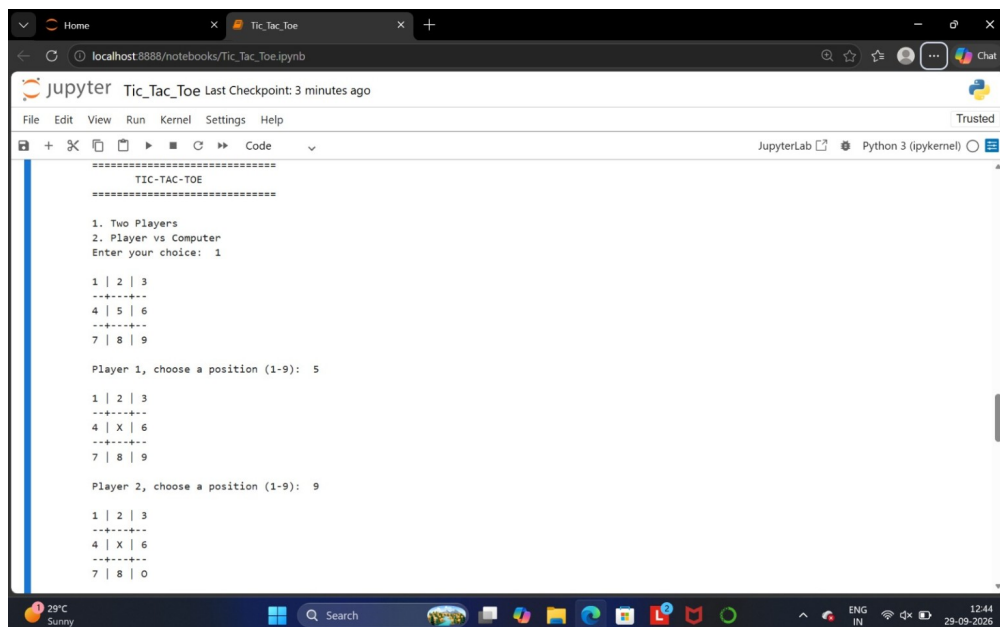
print("\nFinal Board:")
print(board[0], "|", board[1], "|", board[2])
print("-----")
print(board[3], "|", board[4], "|", board[5])
print("-----")
print(board[6], "|", board[7], "|", board[8])

else:
    print("\nInvalid Choice!")

play = input("\nDo you want to play again? (yes/no): ")

print("\nThank you for playing!")
```

Figure 5. Project screenshot from Jupyter Notebook.



```
=====
TIC-TAC-TOE
=====

1. Two Players
2. Player vs Computer
Enter your choice: 1

1 | 2 | 3
---
4 | 5 | 6
---
7 | 8 | 9

Player 1, choose a position (1-9): 5

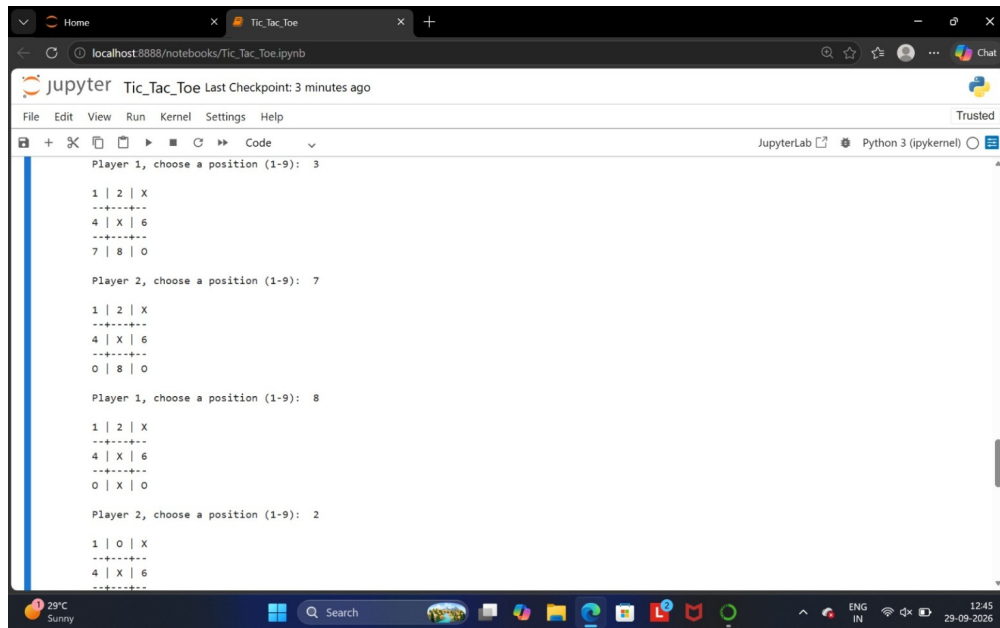
1 | 2 | 3
---
4 | X | 6
---
7 | 8 | 9

Player 2, choose a position (1-9): 9

1 | 2 | 3
---
4 | X | 6
---
7 | 8 | O
```

Figure 6. Project screenshot from Jupyter Notebook.

Screenshots and Results



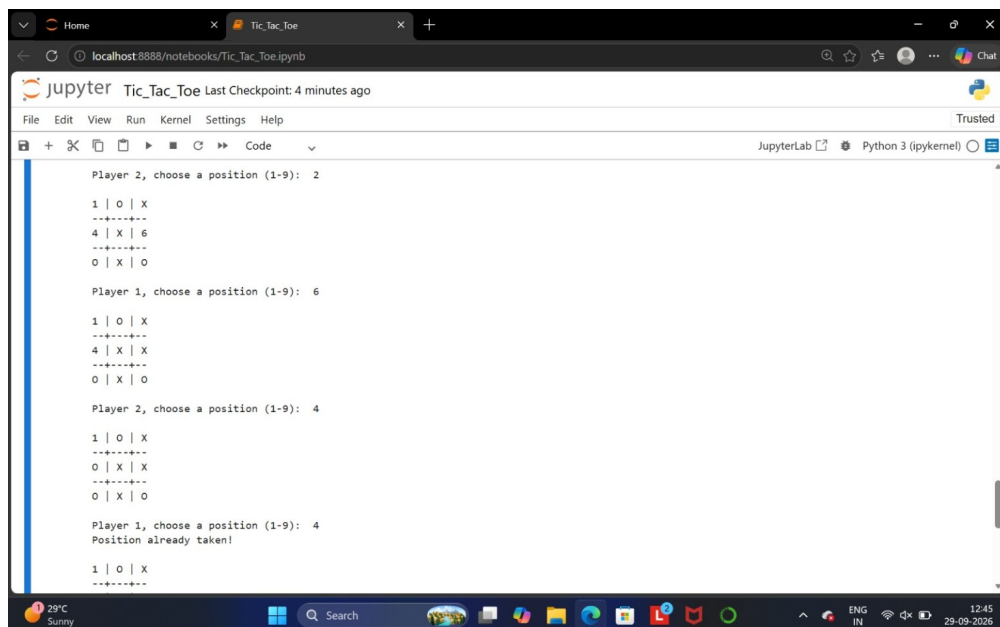
```
Player 1, choose a position (1-9): 3
1 | 2 | X
-----
4 | X | 6
-----
7 | 8 | 0

Player 2, choose a position (1-9): 6
1 | 2 | X
-----
4 | X | 6
-----
0 | 8 | 0

Player 1, choose a position (1-9): 8
1 | 2 | X
-----
4 | X | 6
-----
0 | X | 0

Player 2, choose a position (1-9): 2
1 | 0 | X
-----
4 | X | 6
-----
```

Figure 7. Project screenshot from Jupyter Notebook.



```
Player 2, choose a position (1-9): 2
1 | 0 | X
-----
4 | X | 6
-----
0 | X | 0

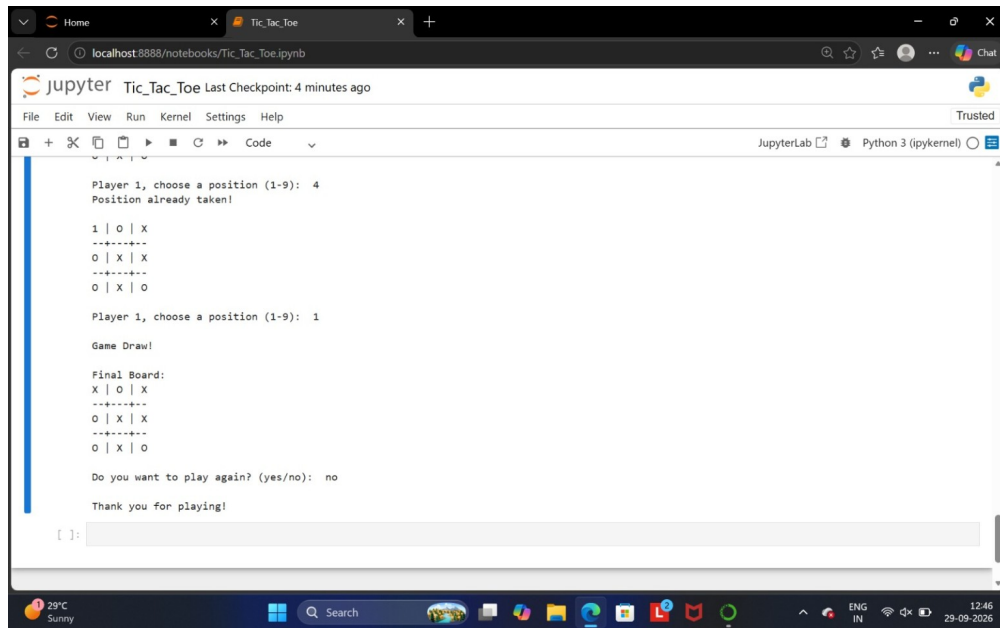
Player 1, choose a position (1-9): 6
1 | 0 | X
-----
4 | X | X
-----
0 | X | 0

Player 2, choose a position (1-9): 4
1 | 0 | X
-----
0 | X | X
-----
0 | X | 0

Player 1, choose a position (1-9): 4
Position already taken!
1 | 0 | X
-----
```

Figure 8. Project screenshot from Jupyter Notebook.

Screenshots and Results



```
Player 1, choose a position (1-9): 4
Position already taken!

 1 | O | X
---+---
 0 | X | X
---+---
 0 | X | O

Player 1, choose a position (1-9): 1
Game Draw!

Final Board:
X | O | X
---+---
O | X | X
---+---
O | X | O

Do you want to play again? (yes/no): no
Thank you for playing!
```

Figure 9. Project screenshot from Jupyter Notebook.